



메뉴바를 클릭하면  
계정을 관리할 수 있어요

논문 리뷰

## [paper] GAN 논문 리뷰 : Generative Adversarial Nets

letterhill | 2026. 5. 24. 23:56 ⓘ

### Abstract

본 논문에서는 두 개의 Adversarial한 모델 두 개를 동시에 학습하는 방법을 제안한다  
G는 생성하는 모델로, 데이터의 분포를 캡처하고  
D는 discriminative, 즉 판별하는 모델로 샘플이 G가 만든 가짜 데이터인지, 진짜 데이터인지 판별한다.

### Introduction

DL에서 성공적인 모델은 Discriminative Model, 즉 판별 모델이었다.  
어떠한 이미지가 입력으로 들어왔을 때, 이 이미지는 고양이이다. 개다. 이런 식으로 판별을 해주는 모델이다. 본 모델  
들은 Backprop+Dropout 알고리즘 기반이며, 특히 ReLU와 같은 Piecewise linear units으로 인해 gradient도 안  
정적으로 수렴하여 매우 뛰어난 성능을 보였다.  
하지만 Discriminative Model에 비해, **Generative Model**은 아래와 같은 이유로 발전이 뒤쳐졌다.

#### 1) Interactable Probability

- 생성 모델은 데이터가 주어졌을 때, 그 데이터를 만들어낸 진짜 확률 분포  $p_x$ 를 학습한다. 여기서 MLE를 쓰려면

$$\theta^* = \arg\max_{\theta} \sum_i \log p_{\theta}(x_i)$$

$p_{\theta}(x)$ 를 계산해야 하는데, 여기서 보통 잠재변수  $z$ 를 도입한다.

$$p_{\theta}(x) = \int p_{\theta}(x | z) p(z) dz$$

문제는 이 적분인데,  $z$ 가 고차원이면 적분 범위가 너무 커지고, 근사를 써도 오차가 크거나 computation cost가 너무 크기 때문이다.

#### 2) ReLU 활용 어려움

Discriminative 모델에서는 출력이 클래스 확률로 스칼라나 벡터이므로 음수를 0으로 잘라내도 ReLU 분류 정보는 그대로 남아있다. 하지만 Generative 모델에서는 간단한 클래스 확률을 출력하는 것이 아니라 전체 데이터 distribution을 표현하기 때문에 ReLU를 사용하기 어렵다.

그래서 제안된 모델: **Adversarial Nets**

Generative 모델: 화폐 위조범 -> 진짜 화폐처럼 위폐를 만들어 경찰을 속이는 게 목표

Discriminative 모델: 위조 화폐를 잡아내는 경찰 -> 진짜와 위폐를 구분해내는 게 목표

G와 D가 Adversarial하게 학습을 하는 신경망이다.

메뉴바를 클릭하면  
계정을 관리할 수 있어요

## Related work - 기존 모델들의 Limitations

### 1. Undirected Graphical Models

ex) RBM, DBM

- 확률을 비정규화된 potential functions의 곱으로 표현

-  $Z = \text{Partition Function}$  전체 상태에 대한 *summation/integration*로 정규화

=> 확률을 구하려면 전체 합은  $Z$ 가 필요한데,  $x$ 가 이미지라면 가능한 경우의 수가 너무 많기 때문에 그 전체 합을 구하는 것이 불가능하다.

### 2. DBN *Deep Belief Networks* - 하이브리드 모델

- 구조: 위쪽 1개의 층은 Undirected 1+아래의 여러 층은 Directed

-> 두 방식의 단점을 모두 가진다.

### 3. Log-likelihood를 직접 쓰지 않는 방법들

ex) Score Matching, NCE

-  $z$ 를 안 쓰기 위해 제안된 방법들이다.

- 학습된 probability density를 수학적으로 명확하게 표현할 수 있어야 하지만  $z$ 가 여러 층인 복잡한 모델은 수식 자체를 쓰지 못한다는 문제가 있다.

- 특히나 NCE에서는 G 모델은 고정된 noise 분포를 사용하여 어느 정도 학습 이후에는 성능 개선이 어렵다는 추가 문제가 있다.

### 4. GSN :Generative Stochastic Network

- 샘플을 여러 스텝에 걸쳐 생성하는 모델이다.

- 그리고, 각 스텝의 출력이 다음 스텝의 입력으로 전달되는데 여기서 ReLU를 쓰면 스텝마다 커져서 폭발한다는 문제가 있다.

## Adversarial nets

introduction에서 설명한대로 GAN에서는 두 개의 모델을 사용한다.

Generator G: 가짜 데이터를 만드는 생성자

Discriminator D: 진짜/가짜를 구분하는 판별자

- G: 노이즈  $z$ 를 입력으로 받아서 -> 가짜 이미지  $Gz$ 를 생성
- D: 이미지  $x$ 를 입력으로 받아서 -> 진짜일 확률  $Dz$ 를 출력

## Objective Function

GAN의 목적 함수는 아래와 같다.

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

Eq 1

메뉴바를 클릭하면  
계정을 관리할 수 있어요

terms을 분리해서 살펴보자면

1  $\mathbb{E}[\log D(x)]$

- 진짜 데이터  $x$ 가 들어오면  $D(x)$ 가 1에 가까울 수록  $\log$ 값이 커진다.

=> D입장에서는 이 값을 최대화하고 싶어한다. 진짜데이터가들어오면커지게!

2  $\mathbb{E}[\log 1 - D(G(z))]$

- 가짜 데이터  $Gz$ 가 들어오면  $D(G(z))$ 가 0에 가까울수록  $\log$ 값이 커진다.

- D 입장: 가짜 데이터가 들어오면 0으로 출력하려고 하니까 maximize하는 게 목표 = 가짜를 보고 가짜라고 판단!

- G 입장:  $D(G(z))$ 를 1에 가깝게 만들어 이 term을 minimize하는 게 목표! = D가 가짜를 보고 진짜라고 판단!

D: V를 maximize하는 것이 목표 -> 진짜는 1, 가짜는 0

G: V를 minimize하는 것이 목표 -> 가짜를 1로 출력

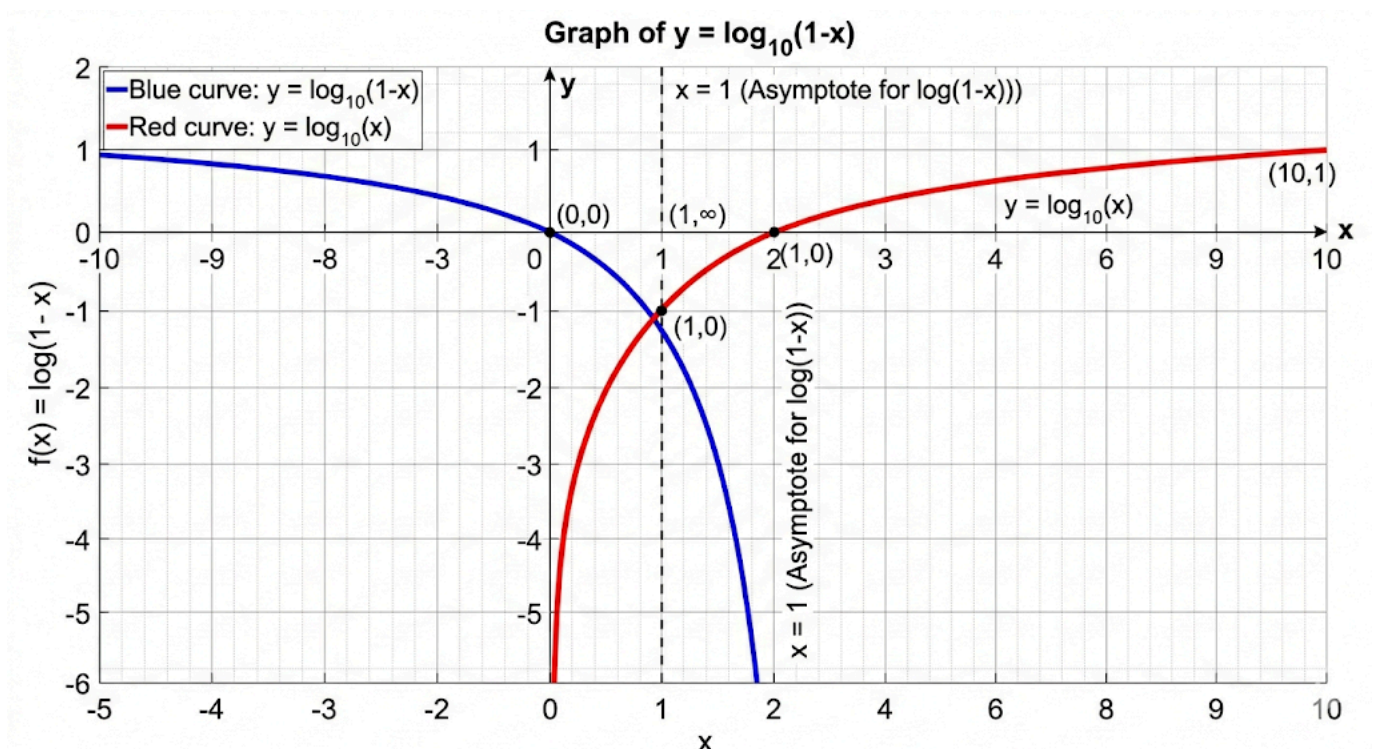
-> minimax game

D를 k번 반복한 뒤, G를 최적화 *update*하는 방식으로 학습한다.

또한, Eq1의 경우 초기에 학습이 제대로 되지 않아 수렴이 어려울 수 있다는 문제점이 있다.

가짜 이미지를 만드는 G의 성능이 너무 좋지 않으면 D가 가짜를 쉽게 구별해버려  $DG(z)$ 가 거의 0에 수렴한다.

$\log 1 - z$  그래프는 아래의 파란색 그래프처럼 0근처에서 기울기가 flat하므로 그 결과 G가 학습을 하지 못하게 된다.



따라서 목적 함수 그래프를 flip해버리는 방식으로 해결한다.

| 이전파란색그래프                      | solution빨간색그래프          | 메뉴바를 클릭하면 계정을 관리할 수 있어요 |
|-------------------------------|-------------------------|-------------------------|
| $\log 1 - D(G(z))$ 를 minimize | $\log DG(z)$ 를 maximize |                         |
| 초반 gradient flat으로 학습 진행 더딤   | 초반 gradient 커서 학습이 잘 됨  |                         |

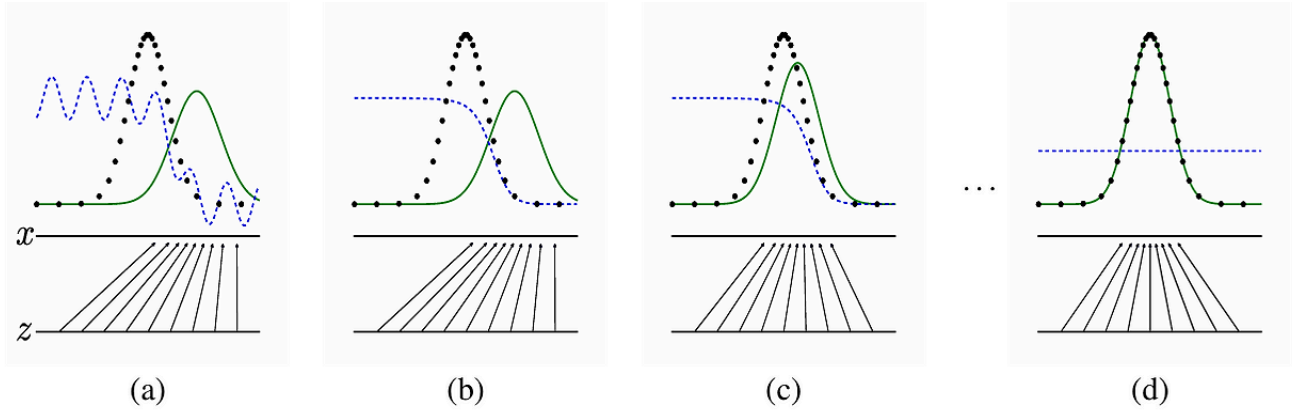


Fig 1

- a 초기: g와 data의 분포가 다르고, D도 불안정
- b D 학습 후: D의 성능이 좋아져서 파란선이 안정
- c G 학습 후: p\_g가 p\_data 쪽으로 이동
- d 최종: p\_g=p\_data -> D가 구별 불가

## Theoretical Results

이 장에서는 총 세 단계를 통해 수식적인 증명을 한다.

### 4.1 Global Optimality

Step 1: 최적의  $D^*$  찾기

G가 고정값이라고 가정했을 때, D의 Objective function:

$$V(G, D) = \int_x p_{data}(x) \log D(x) + p_g(x) \log(1 - D(x)) dx$$

적분을 각 x값에 대한 합이라고 생각하면 x를 고정했을 때,  $p_{data}x=a$ ,  $p_gx=b$ ,  $Dx=y$ 로 변환할 수 있다.

그러면 식도 아래와 같이 변환할 수 있다.

$$f(y) = a \log y + b \log(1 - y)$$

이렇게 되면, 최적화할 값은 y 하나뿐이기 때문에, y에 대해 미분하여 최대값을 찾을 수 있다. a, b는 상수

$$V(G, D) = \int_{\mathbf{x}} \underbrace{p_{\text{data}}(\mathbf{x})}_a \underbrace{\log D(\mathbf{x})}_y + \underbrace{p_g(\mathbf{x})}_b \log(1 - D(\mathbf{x})) d\mathbf{x}$$

메뉴바를 클릭하면  
계정을 관리할 수 있어요

$$f(y) = a \log y + b \log(1 - y)$$

$$\frac{df}{dy} = \frac{a}{y} - \frac{b}{1-y} = 0$$

$$a(1-y) = by$$

$$a = ay + by = y(a+b)$$

$$\Rightarrow y^* = \frac{a}{a+b}$$

대입 치환 ↪

$$D(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})}$$

Step 2: CG

최적  $D^*$ 를  $V$ 에 대입하여  $G$ 만의 objective function  $CG$ 를 만든다.

$$C(G) = \max_D V(G, D)$$

=>  $D^*$  대입

$$C(G) = \max_D V(G, D)$$

$$= \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D_G^*(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_g} [\log(1 - D_G^*(G(\mathbf{z})))]$$

$$= \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D_G^*(\mathbf{x})] + \mathbb{E}_{\mathbf{x} \sim p_g} [\log(1 - D_G^*(\mathbf{x}))]$$

$$= \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \left[ \log \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})} \right] + \mathbb{E}_{\mathbf{x} \sim p_g} \left[ \log \frac{p_g(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})} \right]$$

Eq 4

Step 3:  $CG \rightarrow$  JSD 변환

분모에 2를 곱하고 나누는 수학적 trick을 적용하여 다음과 같이 식을 바꾼다.

$$C(G) = -\log(4) + KL \left( p_{\text{data}} \left\| \frac{p_{\text{data}} + p_g}{2} \right\| \right) + KL \left( p_g \left\| \frac{p_{\text{data}} + p_g}{2} \right\| \right)$$

Eq 5

$$C(G) = -\log(4) + 2 \cdot JSD(p_{\text{data}} \| p_g)$$

## 4.2 Convergence of Algorithm 1

메뉴바를 클릭하면  
계정을 관리할 수 있어요

G와 D가 충분한 capacity를 갖고 매 스텝 D가 optimal한 D\*에 도달하면, p\_g는 p\_data로 수렴한다.

**Algorithm 1** Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator,  $k$ , is a hyperparameter. We used  $k = 1$ , the least expensive option, in our experiments.

**for** number of training iterations **do**

**for**  $k$  steps **do**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Sample minibatch of  $m$  examples  $\{x^{(1)}, \dots, x^{(m)}\}$  from data generating distribution  $p_{\text{data}}(x)$ .
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[ \log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

**end for**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

**end for**

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Algorithm 1은 D와 G를 번갈아 업데이트한다.

\* D를 k번 업데이트하고 G 한 번 업데이트

그렇다면 D랑 G를 번갈아 업데이트 하면 p\_g = p\_data에 수렴할 수 있을까?

에 대해서 4-2에서 증명한다.

증명의 전제 조건

조건 1) G와 D가 충분한 capacity를 가짐

조건 2) D를 k번 업데이트할 때 완벽히 수렴한다고 가정

4-1에서 계산한 V를 p\_g의 함수로 보면:

$$U(p_g, D) = V(G, D)$$

여기서 핵심은  $U(p_g, D)$ 는  $p_g$ 에 대해 Convex하다는 것이다.

## Experiments

## Settings

### 1) 데이터셋

- MNIST: 손글씨 숫자 이미지
- TFD: 얼굴 이미지
- CIFAR-10: 자연 이미지

메뉴바를 클릭하면  
계정을 관리할 수 있어요

### 2) 네트워크 구조

|              | Generator G  | Discriminator D |
|--------------|--------------|-----------------|
| activation f | ReLU+Sigmoid | Maxout          |
| Dropout      | 사용X          | 사용              |
| 노이즈 input    | 맨 아래 층에만     |                 |

| G에 입력층에만 noise를 넣어 단순하게 유지한다.

### 3) 성능 평가: Gaussian Parzen Window

1. G로 샘플 여러 개 생성
2. 샘플들 주변에 Gaussian Distribution을 씌운다.
3. 테스트 데이터가 그 분포 아래서 얼마나 높은 확률을 갖는지 측정  
-> log likelihood

## 결과

| Model            | MNIST                         | TFD                             |
|------------------|-------------------------------|---------------------------------|
| DBN [3]          | 138 $\pm$ 2                   | 1909 $\pm$ 66                   |
| Stacked CAE [3]  | 121 $\pm$ 1.6                 | <b>2110 <math>\pm</math> 50</b> |
| Deep GSN [6]     | 214 $\pm$ 1.1                 | 1890 $\pm$ 29                   |
| Adversarial nets | <b>225 <math>\pm</math> 2</b> | <b>2057 <math>\pm</math> 26</b> |

Table 1

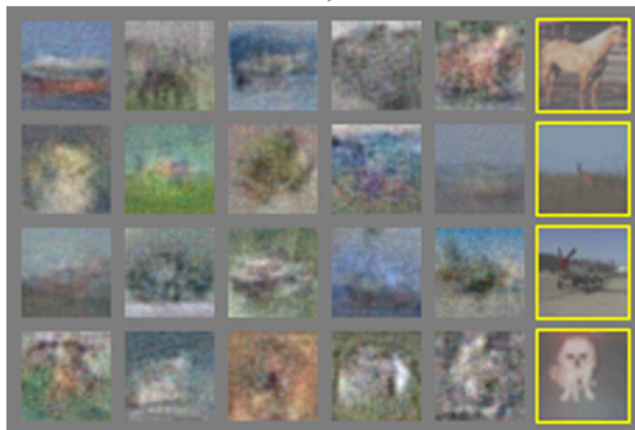
Table 1에서 GAN이 압도적으로 높은 성능을 보인다.



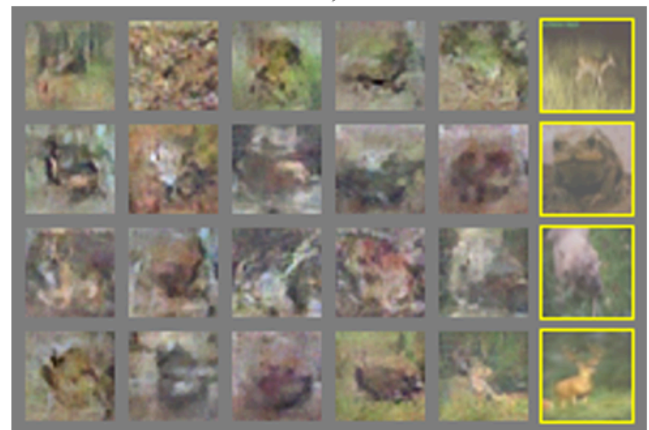
a)



b)



c)



d)

Fig 2

Fig2에서 G가 가짜 이미지를 생성하면 가짜 이미지와 가장 비슷한 진짜 train data\*노란박스)를 고른다 -> GAN이 그냥 train data를 단순 암기 *Memorization*한 게 아닌 것을 증명한단다.



Figure 3: Digits obtained by linearly interpolating between coordinates in  $z$  space of

메뉴바를 클릭하면  
계정을 관리할 수 있어요

|                   | Deep directed graphical models             | Deep undirected graphical models                                                          | Generative autoencoders                                                        |                                                                                |
|-------------------|--------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Training          | Inference needed during training.          | Inference needed during training. MCMC needed to approximate partition function gradient. | Enforced tradeoff between mixing and power of reconstruction generation        | Synchronizing the discriminator with the generator. Helvetica.                 |
| Inference         | Learned approximate inference              | Variational inference                                                                     | MCMC-based inference                                                           | Learned approximate inference                                                  |
| Sampling          | No difficulties                            | Requires Markov chain                                                                     | Requires Markov chain                                                          | No difficulties                                                                |
| Evaluating $p(x)$ | Intractable, may be approximated with AIS  | Intractable, may be approximated with AIS                                                 | Not explicitly represented, may be approximated with Parzen density estimation | Not explicitly represented, may be approximated with Parzen density estimation |
| Model design      | Nearly all models incur extreme difficulty | Careful design needed to ensure multiple properties                                       | Any differentiable function is theoretically permitted                         | Any differentiable function is theoretically permitted                         |

Table 2: Challenges in generative modeling: a summary of the difficulties encountered by different approaches to deep generative modeling for each of the major operations involving a model.

Fig 3, Table 2

## Advantages and Disadvantages

### GAN의 장점

- Markov Chain이 불필요해 빠르고 단순하다.
- $p_x$ 를 명시할 필요가 없다.

### GAN의 단점

- D와 G 중 한 쪽이 너무 강해지면 불안정해지므로 균형을 잘 맞춰야 한다.
- $p_x$ 를 직접 구하지 못해 평가가 어렵다.

## Conclusions and future work

GAN의 발전 방향에 대해 제시한 장이다. 실제로 현재 2026년 기준으로는 본 논문에서 제시한 발전 방향을 구현한 모델들이 다 존재한다.

1) Conditional GAN: G와 D 둘 다에 조건  $c$ 를 추가 입력으로 넣는다. -> cGAN으로 발전

2) Learned Approximate Inference: 보조 네트워크를 훈련시켜  $x$ 가 주어졌을 때  $z$ 를 역추정 -> VAE-GAN, BiGAN으로 발전

3) Conditional 모델 전제 학습: 파라미터를 공유하는 조건부 모델들의 집합을 학습한다 -> Pix2Pix

4) Semi-supervised Learning: 레이블이 있는 데이터가 적을 때, D 또는 inference net이 활용하여 분류 성능을 향상시킨다. -> Semi-supervised GAN

5) Efficiency Improvements

메뉴바를 클릭하면  
계정을 관리할 수 있어요

## <Swin Transformer Intro Review>

CNN 시대에서는 AlexNet이 등장하고 CNN 구조가 점점 발전하며 더 깊고 넓은 모델이 CV의 표준 backbone이 되었다.

하지만 NLP에서는 Transformer가 등장하며 LSTM에도 탁월하다는 장점이 있어 아주 큰 성능 향상과 발전이 있었다.

-> "Transformer 모델을 Computer Vision에도 도입할 수 있을까?"라는 질문에서 본 논문은 시작한다.

Transformer를 비전에 그대로 적용하면 **어떤 문제**가 생길까?

### 1) Scale 문제

NLP 토큰은 단어 하나가 고정된 크기이지만, 비전 토큰은 물체가 이미지마다 크기가 다 다르기 때문에 고정 scale token으로 처리하기 어렵다.

### 2) Intensity 문제

- NLP: 문장 = 수백 개 토큰

- 이미지: 픽셀 수 = 수십만 개

기존 ViT의 self-attention 복잡도는  $O(n^2)$ 으로 이미지 크기가 커지면 폭발적으로 커진다.

Swin Transformer의 solution!

### 1) Hierarchical 구조

- 기존 ViT: 16x16 해상도로 고정

- Swin T: hierarchical한 해상도로 4->8->16->32 점점 커진다.

### 2) Shifted Window Self-Attention

- 기존 self-attention은 모든 픽셀끼리 계산하여  $O(n^2)$  복잡도였다.

- Swin의 self-attention: 윈도우 안에서만 계산한다.

하지만 윈도우끼리 독립적이면 정보 교류가 없는 문제를 Shifted Window로 해결하여 이전 층의 윈도우 경계를 다음 층에서 넘어서 연결함으로써 윈도우 간 정보 교류를 가능하게 한다.

Sliding window에 비해 key set을 고려하기 때문에 메모리를 효율적으로 사용한다.

메뉴바를 클릭하면  
계정을 관리할 수 있어요

'논문리뷰' 카테고리의 다른 글

|                                                                                                   |         |            |
|---------------------------------------------------------------------------------------------------|---------|------------|
| <a href="#">Denoising Diffusion Probabilistic Models</a>                                          | 0       | 2026.05.17 |
| <a href="#">[paper] Efficient Memory Management for Large LanguageModel Serving with PagedAt</a>  | 0       | 2026.05.11 |
| <a href="#">[paper] AN IMAGE IS WORTH 16X16 WORDS:TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE</a> | 논문리뷰 0  | 2026.05.05 |
| <a href="#">deep residual</a>                                                                     | 논문 리뷰 2 | 2026.05.02 |
| <a href="#">[NLP] BERT</a>                                                                        | 논문 리뷰 1 | 2026.04.06 |

## '논문 리뷰' Related Articles

NO IMAGE

Denoising Diffusion Probabilistic Models

Parameters (26GB, 65%)

KV Cache (>30%)

Others

NVIDIA A100 40GB

[paper] Efficient Memory Management for Large...

NO IMAGE

[paper] AN IMAGE IS WORTH 16X16...

NO IMAGE

deep residual 논문 리뷰

## letterhill 님의 블로그

letterhill 님의 블로그입니다.



댓글 0

letterhill

내용을 입력하세요.



등록

메뉴바를 클릭하면  
계정을 관리할 수 있어요